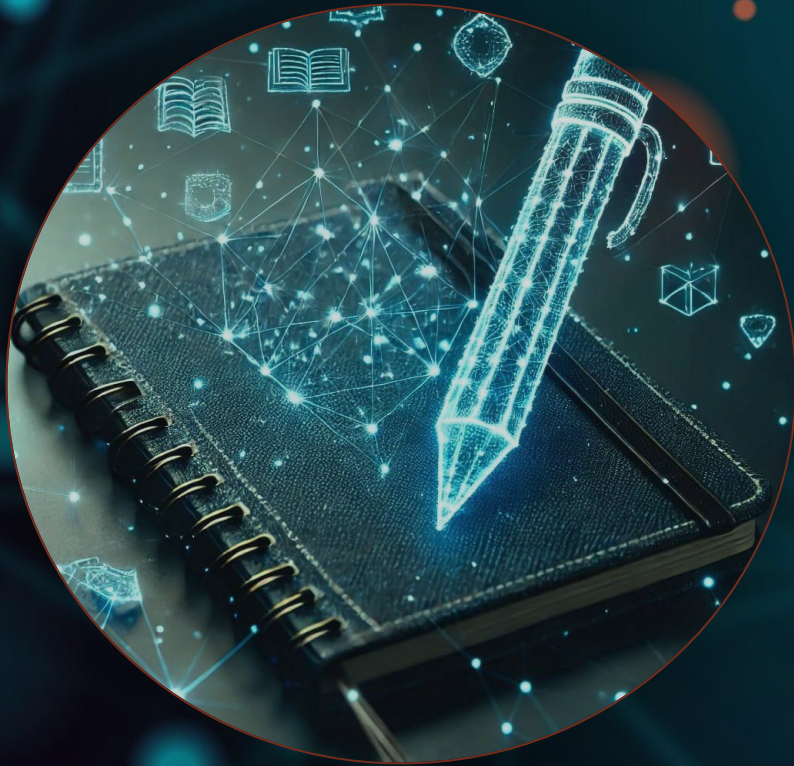




STUDY GROUP FINDER

CB.SC.U4CSE23512

CB.SC.U4CSE23516

CB.SC.U4CSE23520

CB.SC.U4CSE23524

CB.SC.U4CSE23554

CB.SC.U4CSE23557

Problem Statement

In an educational environment where collaborative learning is emphasized, there is a need for an efficient study group management system. This system aims to facilitate the formation of study groups based on **subjects, individual student performance, and preferred languages**. The goal is to ensure that groups maintain a balanced composition of students across different performance categories, enabling effective peer-to-peer learning while allowing for flexibility in group dynamics based on feedback.

Solution

In modern educational environments, effective collaboration among students is essential for enhancing learning experiences. Traditional grouping methods often overlook individual performance levels and preferences, resulting in suboptimal study sessions. This study group management system employs a hashmap approach(optimal), where subjects act as keys, and the corresponding values hold another hashmap (key: languages; values: linked list of student groups categorized by academic performance).

Each group is structured to maintain a balance of students across three performance tiers: low, middle, and high scores. Groups must include more than two high-achieving students, a maximum of three middle performers, and no more than two low scorers. This design encourages peer learning while reducing discouragement for lower-performing students.

A feedback mechanism gathers ratings from group members after each study session, stored in lists to facilitate average score calculations. If the average rating drops below a predefined threshold, indicating dissatisfaction, the group is reshuffled after two sessions to foster a more productive learning environment.

The system adeptly manages changes in group dynamics. When a high-scoring student leaves, remaining members can wait for a replacement, ensuring stability. This innovative approach adapts to student's evolving needs, promoting both academic achievement and collaboration.

Objectives

Enhance Collaboration: Facilitate collaboration among students with diverse skills and backgrounds to foster a productive learning environment.

Gather Feedback: Collect and analyze feedback from group members to improve the overall study experience and adjust group compositions as needed.

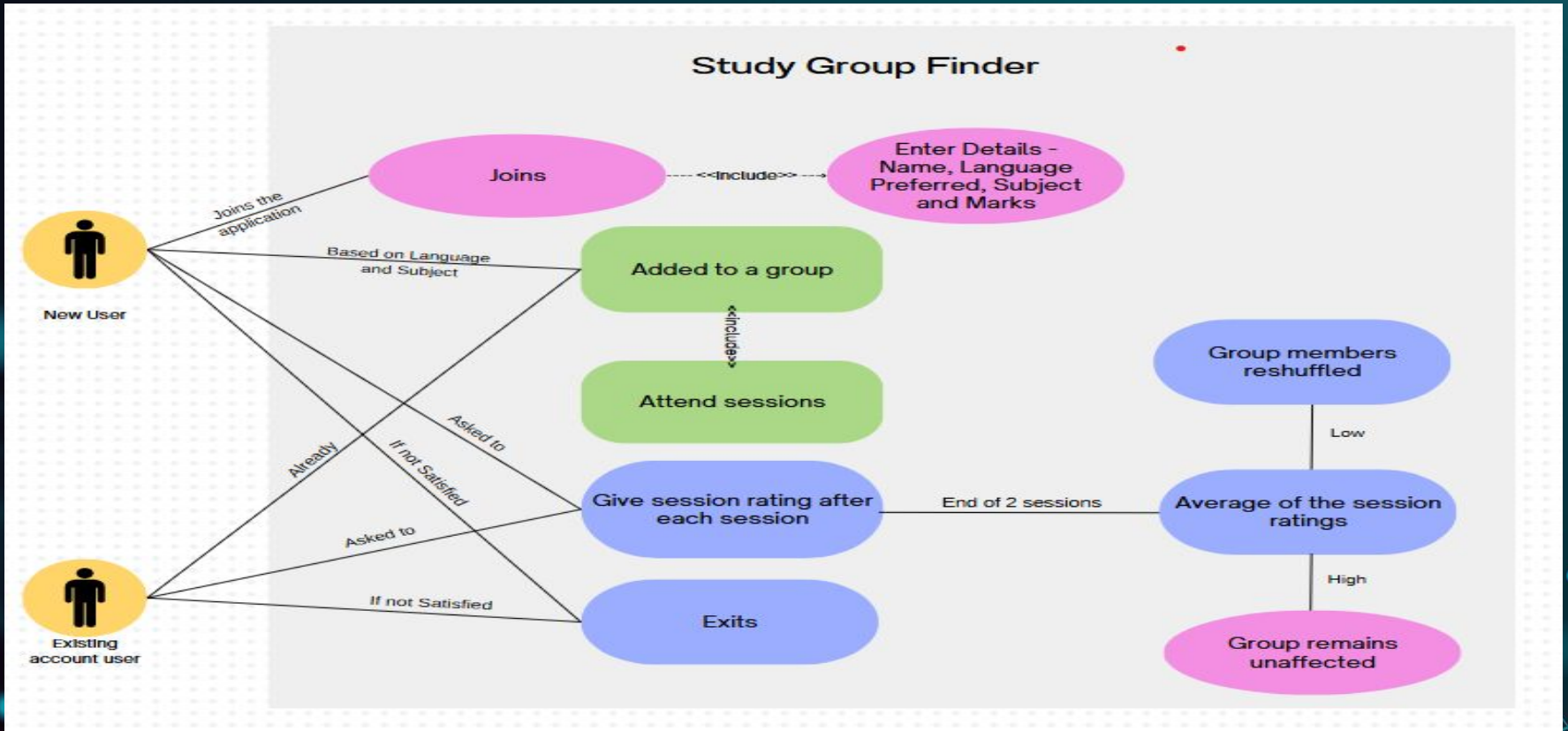
Ensure Balance in Group Composition: Maintain a balanced mix of low, middle, and high-scoring students to maximize learning opportunities and peer support.

Efficient Group Management: Utilize hash maps and linked lists to enable quick access and management of student groups based on performance criteria.

Dynamic Reshuffling: Implement algorithms to dynamically reshuffle groups based on student feedback, ensuring optimal group compositions.

Monitor Group Dynamics: Implement mechanisms to track group performance and student satisfaction, allowing for timely interventions when necessary.

Use Case Diagram



Relevance of the problem statement in the real world - Advantages

Enhancing Education: Addressing common challenges in individual learning, leading to better academic outcomes.

Fostering Collaboration: Promoting teamwork and social connections among students, which are essential in both education and professional environments.

Improving Accountability: Helping students stay on track with their studies, reducing procrastination, and enhancing motivation.

Supporting Diverse Learning Needs: Providing tailored support to students based on their performance and feedback, making education more inclusive (Language based group too).

Teamwork and Communication: Helps students develop essential soft skills like teamwork, communication, and conflict resolution, valued in future workplaces.



Time Complexity

<u>MODULES</u>	<u>TIME COMPLEXITY</u>
SUBJECTS	$O(n)$
LANGUAGES	$O(n)$
WAITINGLIST	$O(n)$
WAITINGLISTNODE	$O(1)$
MEMBER	$O(n)$
MEMBERLISTNODE	$O(1)$
GROUPS	$O(n^2)$
GROUPLISTNODE	$O(n)$

Edge Cases

- **Edge Case:** If there are no students on the waiting list for a group (initially).

Solution: Check for the existence of students in waiting lists before attempting to create a group or allocate members.

- **Edge Case:** If only a subset of group members provides ratings, the calculated average rating might not accurately reflect group performance.

Solution: Implement logic to track the number of ratings received and calculate average rating per session only based on the ratings submitted.##

Edge Cases

- **Edge Case:** If three or more members exit, the group could become unbalanced or fail to meet the required composition for effective learning.

Solution: Implement logic to check the group's composition after members leave. If the group cannot meet its required thresholds, either add new members from the waiting list, else, merge with another compatible unbalanced group.

- **Edge Case:** If a mid-level scorer improves to a high scorer level, it may disrupt the intended distribution of group scores.

Solution: Convert the group into a custom group so that it won't be reshuffled.

- **Edge Case:** If one member consistently rates the group poorly despite high ratings from others, this may indicate underlying issues.

Solution: Exit from the group (interest based) and rejoin if he/she like the system

Edge Cases

- **Edge Case:** If a reshuffle does not result in any changes to the group composition, it may be redundant.

Solution: The chances of this happening with a pseudorandom function are low.

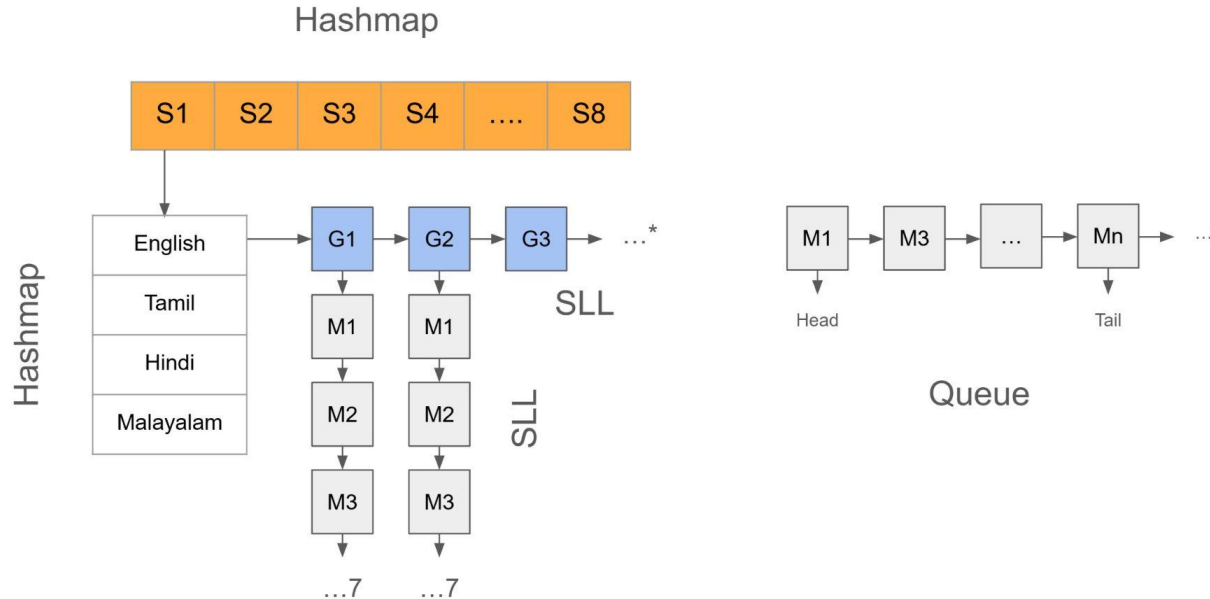
- **Edge Case:** Dequeueing from an empty waiting list.

Solution: The method should return None or an indication that the list is empty.

- **Edge Case:** No session ratings present (empty list).

Solution: The method should, return an indication that no ratings exist.

WHAT DATA STRUCTURES ARE USED ?



Hashtable :

Purpose: To store subjects and languages with easy and fast retrieval.

Why Used: Hash table provide constant-time complexity $O(1)$ for insertions, deletions, and lookups.

In this system :

Subjects Hashtable: Stores subjects as keys, each mapping to another hashtable for languages.

Languages Hashtable: Each subject's language map uses languages as keys, with each language holding a linked list of groups.

Significance: This multi-level hashing structure enables quick access to specific subjects and languages without needing to traverse arrays or lists. It also avoids the challenges of using an array where the indices would not directly correlate to specific subjects or languages. Hash maps offer a straightforward way to organize and look up complex, multi-dimensional data like subjects and languages.

Linked Lists :

Purpose: Used to organize groups and members within each language.

- **Why Used:** Linked lists allow flexible memory usage and efficient insertion and deletion, which are crucial in dynamically managing groups.

They are especially suited for scenarios where the number of groups or members can fluctuate :

Groups Linked List: Each language's hashmap value is a linked list of group nodes. This enables easy addition, removal, and reshuffling of groups without Re-Allocating or resizing memory as in an array..

Members Linked List: Each group maintains a linked list of member nodes, where each node stores member details.

Significance: Unlike arrays, linked lists offer direct pointers to the next element, making additions and deletions straightforward. Additionally, since the group and member lists can change size frequently, linked lists provide flexibility by not requiring contiguous memory.

Queues (Implemented with Linked Lists) :

Purpose: To manage waiting lists for students not yet assigned to any group.

- **Why Used:** Each language has three separate queues (low, mid, high scores) to maintain students waiting for assignment based on their scores.

Waiting List Queues : These queues allow for a fair first-in, first-out (FIFO) system for managing waiting students, organized by their scoring category. New students are enqueued at the tail, and when a group is formed, students are dequeued from the head.

Significance: Linked lists are ideal for implementing queues as they allow easy addition at the end and removal from the front, ensuring efficient management of waiting lists. Using queues ensures that students who wait longer are given priority when forming groups.

Python Lists :

Purpose: To maintain ratings of each session conducted by a group.

Why Used: Python lists provide indexed access and are well-suited for storing fixed-size session data.

Session Ratings: Each group maintains a Python list for session ratings, where the index represents the session number and the value is the rating for that session.

Significance: Lists are ideal here because they allow direct indexing, making it easy to retrieve or update ratings for a specific session by its number. Since the list size corresponds to the number of sessions, there is no need for dynamic resizing or extensive modifications.

Future Enhancements

Enhanced Recommendation System

Utilize machine learning to personalize group formations based on student data.

Real-Time Performance Tracking

Display live student performance metrics for immediate insights.

Analytics Dashboard

Visualize group performance metrics to identify trends and improvement areas.

Interactive UI

Develop a user-friendly interface to enhance user engagement and experience.

Automated Notifications and Reminders:

Send alerts for group sessions and status changes to keep students updated.

Enhanced Data Security and Privacy:

Implement robust measures to protect student information and ensure regulatory compliance.

Bibliography

- ChatGPT
- M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*, Hoboken, NJ: Wiley, 2013.
- Claude AI
- University of Maryland, Baltimore County, "The Benefits of Study Groups,"Available: <https://professionalprograms.umbc.edu/blog/the-benefits-of-study-groups/>
- GeeksforGeeks, "Hashing Data Structure,"Available: <https://www.geeksforgeeks.org/hashing-data-structure/>